

COMP9820 Software Project Management

Software Requirements, Agile Software Development Practices with Scrum

Dr. Basem Suleiman

School of Computer Science and Engineering (CSE)



Acknowledgement of Country

I acknowledge the Bidjigal as the Traditional Custodians of the land we are meeting on today. I pay my respects to Elders past and present and recognise the enduring connection of Aboriginal and Torres Strait Islander peoples to Country. I invite us all to reflect on our shared responsibility to honour and respect this land and its stories."

Outline

- Agile Software Development Practices
 - Agile Manifesto, and Principles
 - Scrum; Team Roles, Events, Artefacts, Planning, Estimation & Monitoring
- Agile Development (Software Requirements)
 - User Stories
- Revisit – Git Team practices

Agile Software Development – Values & Principles

Agile Manifesto – Agile Values

“We are uncovering **better ways of developing software** by doing it and helping others do it. Through this work we have come to **value:**”

- **Individuals and interactions** over processes and tools
- **Working software** over comprehensive documentation
- **Customer collaboration** over contract negotiation
- **Responding to change** over following a plan

The items on the left are **more valued** than those at the right

Agile Manifesto: <http://agilemanifesto.org/>

© 2001, the above authors. This declaration may be freely copied in any form, but only in its entirety through this notice.

Agile Principles (1)

1. Our highest priority is to **satisfy the customer** through **early** and **continuous delivery** of **valuable software**
2. **Welcome changing requirements**, even late in development. Agile processes harness change for the customer's competitive advantage
3. **Deliver working software frequently**, from a couple of weeks to a couple of months, with a preference to the **shorter timescale**
4. **Business people** and **developers** must **work together daily** throughout the project

Agile Alliance: <http://www.agilealliance.org>

Agile Principles (2)

- 5. Build projects around **motivated individuals**. Give them the **environment and support** they need, and **trust** them to get the job done
- 6. The most efficient and effective method of **conveying information** to and within a development team is **face-to-face conversation**
- 7. **Working software** is the primary **measure of progress**
- 8. Agile processes promote **sustainable development**. The sponsors, developers, and users should be able to maintain a **constant pace** indefinitely

Agile Alliance: <http://www.agilealliance.org>

Agile Principles (3)

9. **Continuous attention to technical excellence and good design** enhances agility

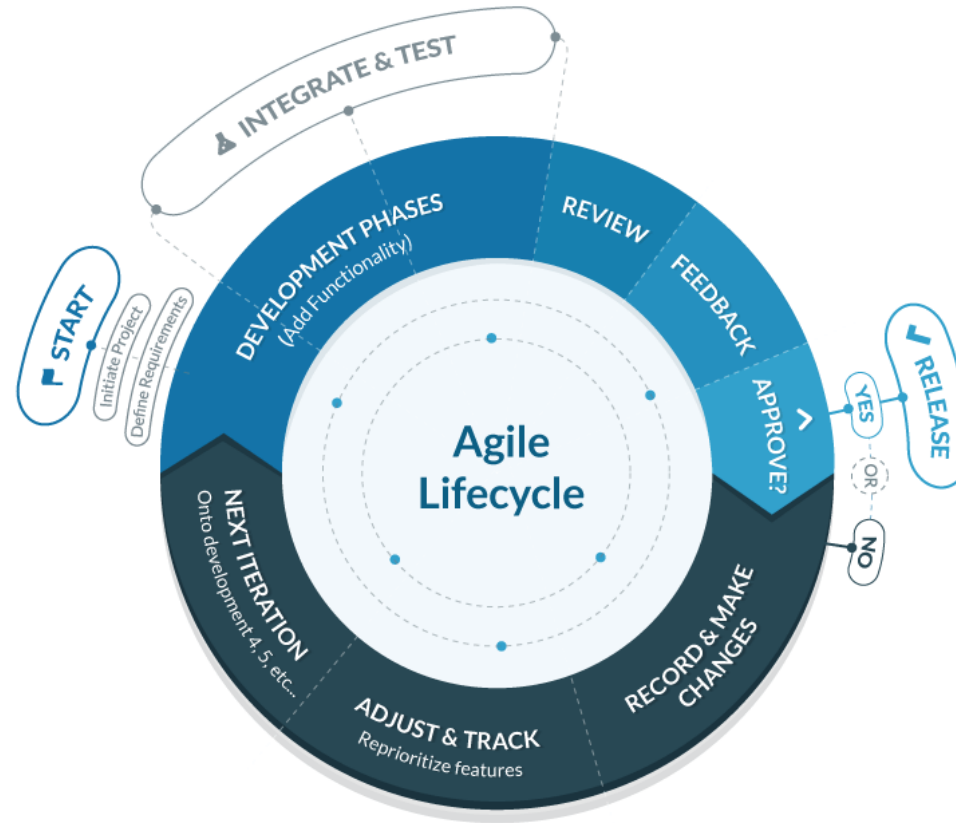
10. **Simplicity** – the art of maximizing the amount of work not done is essential

11. The best architectures, requirements, and designs emerge from **self-organizing teams**

12. At regular intervals, the **team reflects** on how to become more **effective**, then **tunes and adjusts its behavior** accordingly

Agile Alliance: <http://www.agilealliance.org>

Agile Software Development Model



Iterative & Incremental development

Scrum Software Development

An agile method for **managing software development** focused on **delivering products of the highest value**

Teams and roles

- Product Owner, Scrum Master, Dev Team

Events

- Sprint, Sprint Planning, Daily Scrum, Sprint Review, and Retrospective

Artefacts

- Product Backlog, Sprint Backlog, Increment
- Project estimation and Sprint estimation
- Rules govern the relationships between roles, events and artifacts
- All the above components are essential for Scrum success

The Scrum Team

Scrum Team – The Product Owner

Product Owner (PO) maximize the **value** of the **product** and the **work**

- Talks to customer/clients, understand requirements and its priorities
- **Managing the Product Backlog** (only person)
 - Can assign it to the development team, but still accountable

Managing the product backlog:

- Record product backlog items and order it
- Optimize the value of the work the development team performs
- Ensure transparency and clarity of the product backlog
- Ensure the development team understands product backlog

Scrum Team – The Development Team

- **Self-organizing:** turns Product Backlog into increments of potentially releasable functionality
- **Two-pizza team** (4-9 members)
- **Cross-functional:** skills mix necessary to create a product
- Whole team is **accountable (often no sub-teams)**
- Less emphasis on **titles** for Dev team members

Scrum Team – The Scrum Master

Scrum Master (SM) Keeps the team focused on **using Scrum properly**

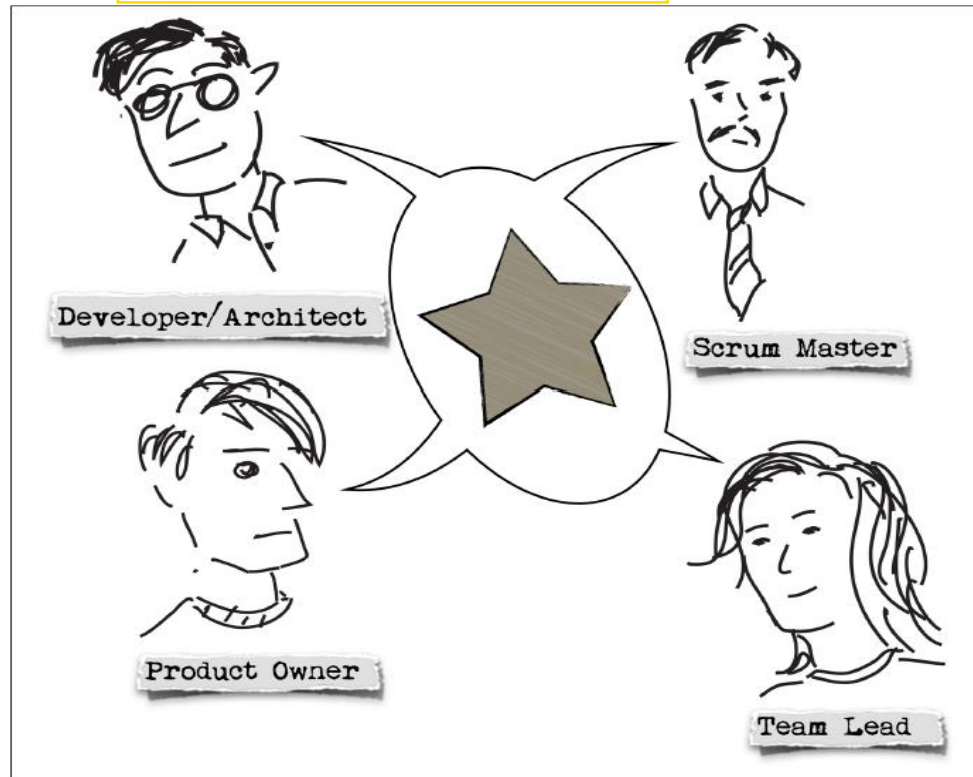
- Everyone **understands Scrum rules** and **values (coaching)**
- Remove roadblocks
- Helps those outside the Scrum team which of their interactions with the Scrum team are/aren't helpful
- **Maximize the value** created by the Scrum team through changing team interactions

Scrum Team

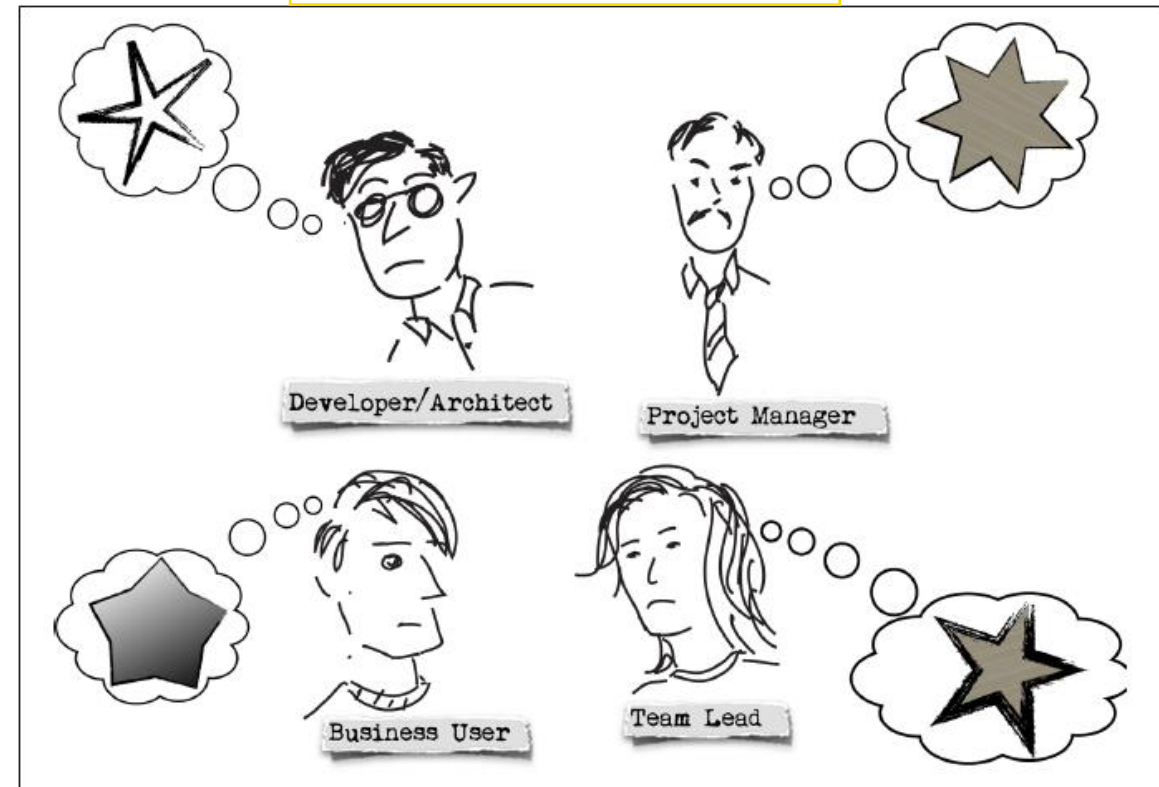
Which structure better describes the Scrum team?



Team A



Team B



The Scrum Events

Scrum Events – The Sprint

A development **iteration** (one cycle)

- Useable and potentially **releasable product increment** is created

Time-boxed (typically 2-4 weeks)

- Too long sprints may lead to changes in the definition

Sprints have **consistent durations** during the product development

Sprint Planning, Daily Scrum (Stand-ups), the Development Work, the Sprint Review and the Sprint Retrospective

Scrum Events – Sprint Planning

Identify the **Sprint Goal** (items from the “Product Backlog”)

Identify **work to be done** to deliver this

Two-parts meeting (SM, PO, and Dev team)

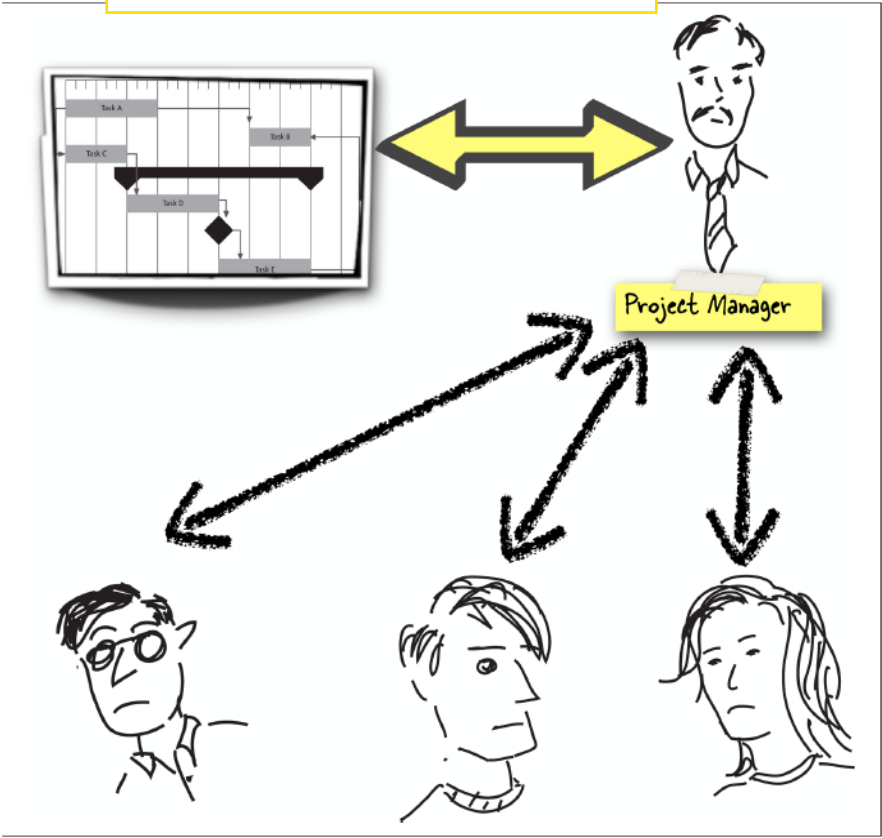
- **Before meeting:** PO prepares prioritized list of most valuable items
- **Meeting part 1:** PO & Dev team select items to be delivered at the end of the sprint based on their value and on the team’s estimate of how much work needed
- **Meeting part 2:** Dev team figure out the individual tasks they’ll use to implement those items

Output: Sprint Backlog

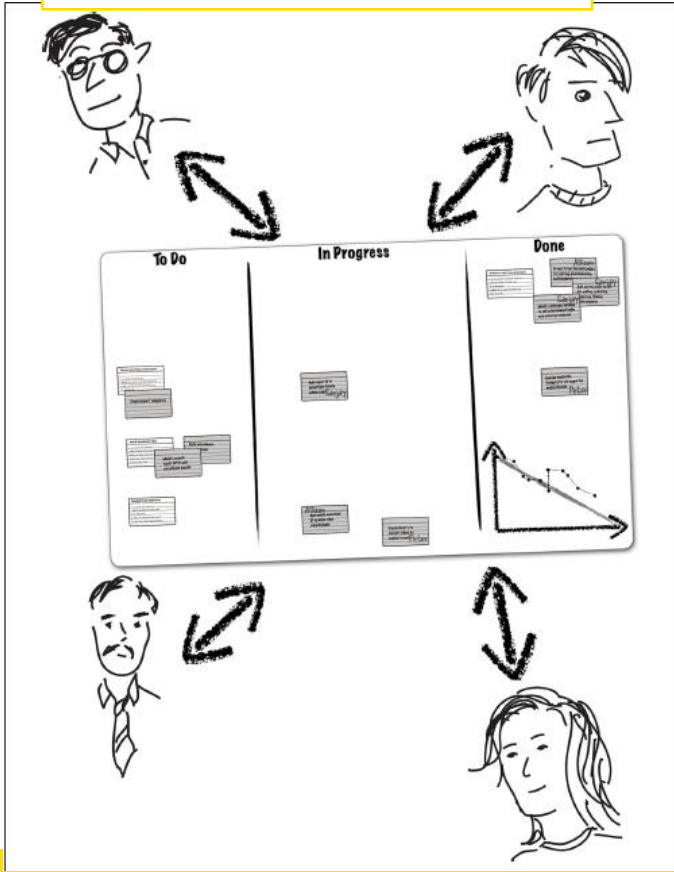
Scrum – Team Structure

Which team organization better describes the Scrum Sprint/iteration planning?

Team C



Team D



Scrum Events – Daily Scrum Meeting (Stand-ups)

- To ensure **problems and obstacles are visible** to the team
- **Timeboxed *15 minutes*** (same time and place each day)
- **All team members** including SM and PO must attend
- Each briefly **answers three questions**:
 - What did I do yesterday that helped the development team meet the Sprint Goal?
 - What will I do today to help the development team meet the Sprint Goal?
 - do I see any obstacles that prevent me or the Dev team to meeting the Sprint Goal?
- **No problem-solving** during the meeting
 - Follow-up meetings if further discussion is required

Scrum Events – Development Work

- Dev team Builds the items in the *Sprint Backlog* into working software
- Should inform the PO if they are overcommitted or can add extra items if time allows
- Must **update the Sprint backlog** and keep it **visible** to everyone

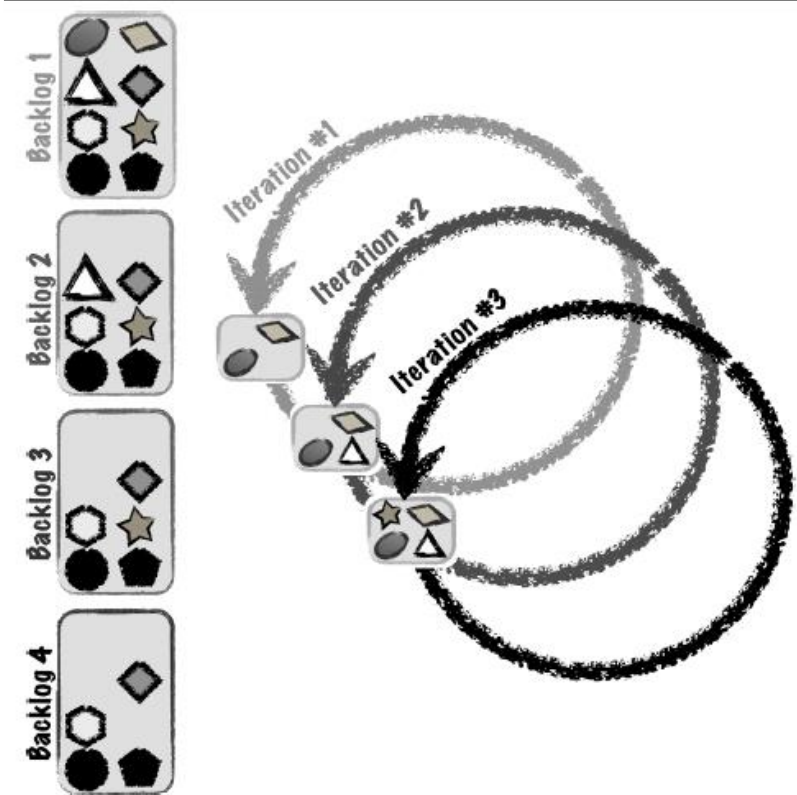
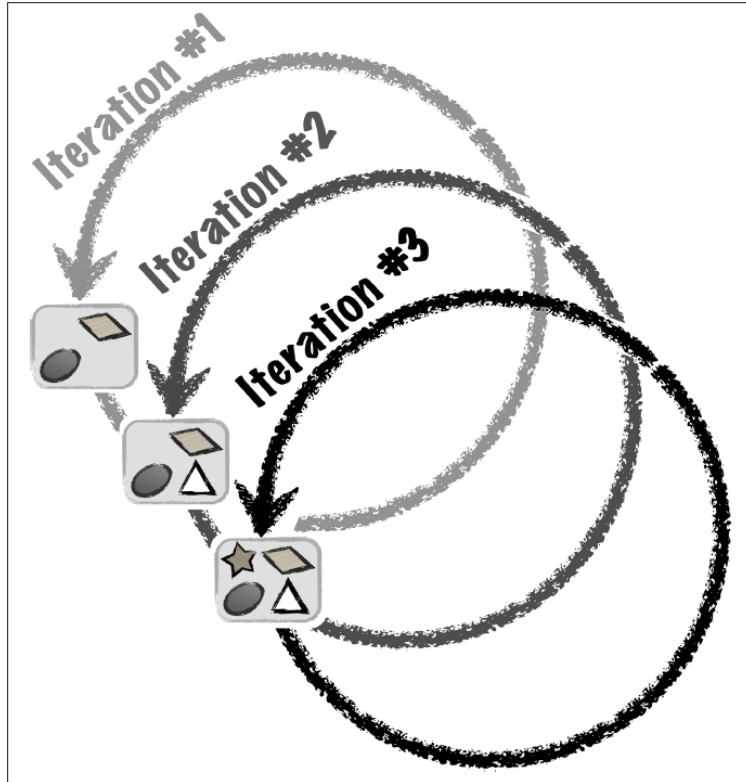
Scrum Events – Sprint Review

Informal meeting at end of the Sprint (max. 4-hours)

- **Dev team demonstrates working software to customers/stakeholders**
 - Items done – completed, tested & accepted by the product owner
 - Only functional working software – not architecture, database design etc.
- **Stakeholders share** their feedback, ideas, feelings, thoughts about the demo
- The Product Owner explains “Done” and “Not Done” items

Output: *revised Product Backlog and probable items for next iteration*

Scrum Iteration (Sprint) Process



* **Product Backlog:** set of all features and sub-features (items) needed to build the product

Scrum Events – Retrospectives

Opportunity for the **Scrum team to inspect itself and create plan for improvements**

- **Inspect** how the last **Sprint** went with regards to **people, relationships, process, and tools**;
- Identify and order the major **items that went well** and **potential improvements**;
- Create a plan for **implementing improvements** to the way the Scrum Team does its work

Scrum Events – Retrospectives

Retrospective meetings (max. 1-2 hours)

The SM and the Dev. team (maybe product owner)

Each person answer **two questions**:

- What went well during the Sprint?
- What can be improved in the future?

The **SM notes improvements** that should be added to the Product Backlog (non-functional items)

- E.g., set-up a better build server, adopting design principles, changing office layout

Output: *identified improvements to be implemented in the next Sprint (adaptation)*

Sprint Retrospective - Example for Sprint 1

*Last
retrospective's
To Try*

First sprint so N/A

What went well

Met sprint goal

What didn't go so well

Duplicated work
(2 team-members
accidentally working on
same task without
knowing)

To Try

Sprint Retrospective - Example for Sprint 1

*Last
retrospective's
To Try*

First sprint so N/A

What went well

Met sprint goal

What didn't go so well

Duplicated work
(2 team-members
accidentally working on
same task without
knowing)

To Try

ensure everyone sends
daily scrum updates if
absent from the daily
scrum meeting
Adam checks that
everyone's present,
and reminds any
absentees
to email status
update to team)

Sprint Retrospective - Example for Sprint 2

<i>Last retrospective's To Try</i>	<i>What went well</i>	<i>What didn't go so well</i>	<i>To Try</i>
Ensure everyone sends daily scrum updates if absent from the daily scrum meeting. (Adam checks that everyone's present, and reminds any absentees to email status update to team) <i>This worked well. Everyone knows what everyone else is doing - no more duplicate work</i>	Met sprint goal No more duplicated work	Software unit demonstrated was buggy	Write unit tests for each unit developed (all members to follow up on that)

Scrum Artifacts

Scrum Artifacts – Product Backlog (PB)

Set of **all features and sub-features** (items) needed to build the product (the “Plan” for multiple iterations)

- Features, functions, requirements, enhancements and fixes from previous Sprints
- Expressed as User Stories

Maintained by the PO in collaboration with customers and team

The source of the **product (software) requirements**

- Evolves over the time and never complete (dynamic)

The **items ordered by priority**

- To deliver value to the customer in each iteration, put the most important things early

Scrum Artifacts – Sprint Backlog (SB)

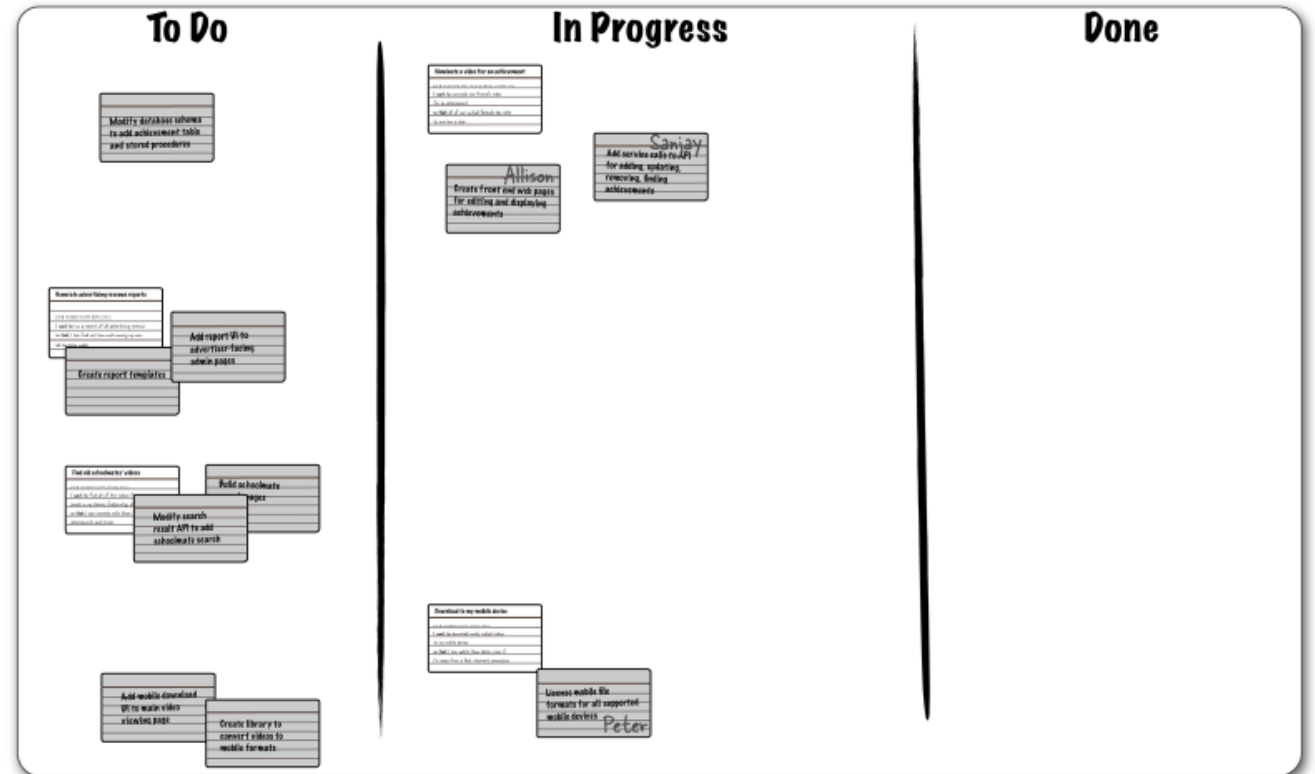
Set of **PB items selected for the Sprint**, and a plan for delivering the product increment and realize the Sprint Goal

- The Dev. team forecasts next items to be implemented
- Includes high-priority improvement identified from previous Sprint
- The Dev. team adds new work to the SB
- The estimated remaining work is updated once an item is completed
- Visible to anyone and to be modified by the Dev. team

Sprint Backlog – Example

Typically divided into 3 sections;
“To Do”, “In Progress”, “Done”

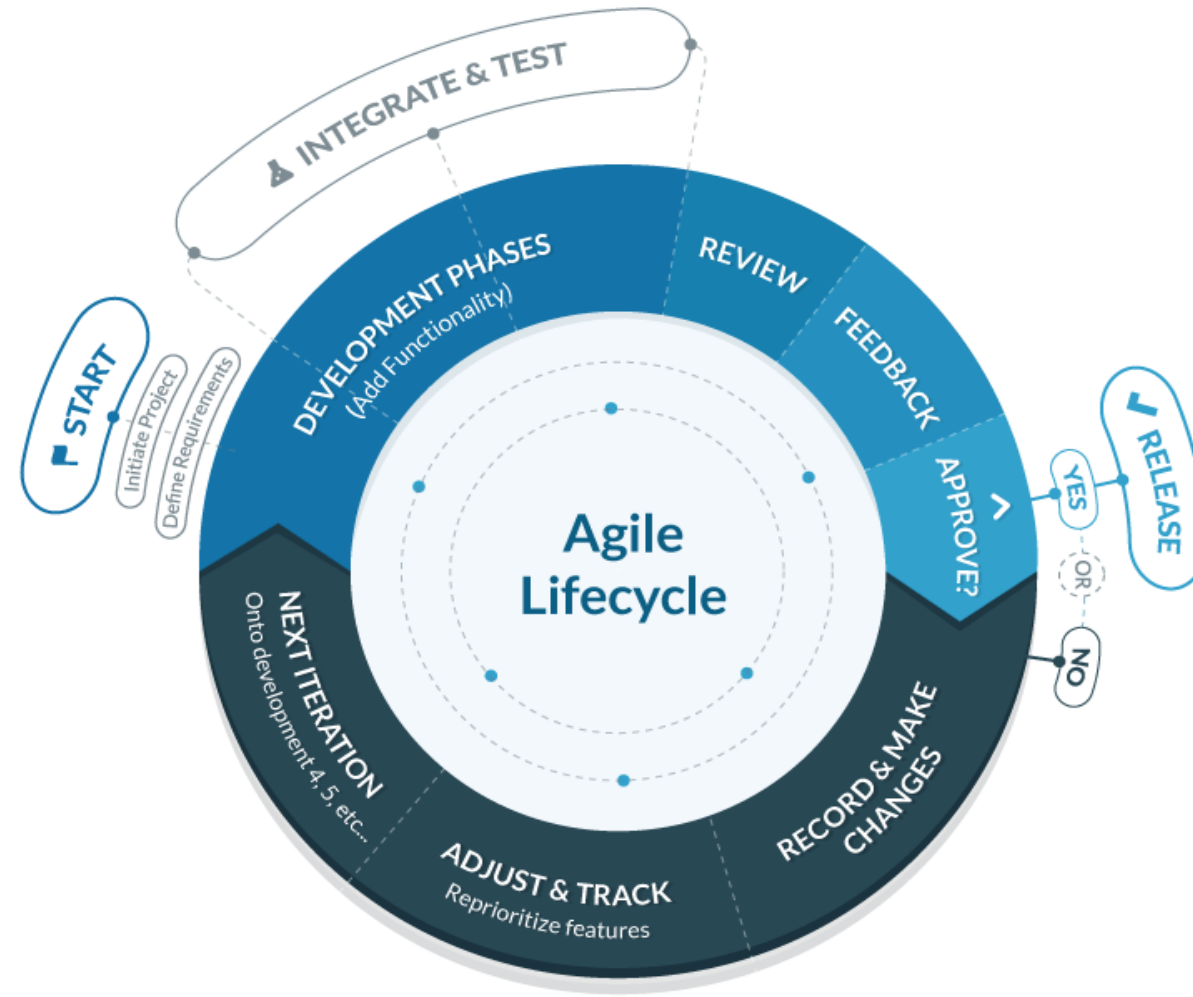
Tools to support Sprint planning and monitoring; e.g., Jira



Agile Planning (Requirements)

User Stories

Requirements – Agile Software Development Model



User Stories

- A method for describing software requirements, tasks or issues
- It helps **agile software development teams** capture a **simplified** and **high-level** description of a **requirement** from an **end user** perspective
- Focuses on **value (benefit)** delivered, not technical implementation
- Agile Development (Planning)
 - Light-weight planning – iterative and incremental
 - Small enough to build and test within a single Sprint
 - Captured in Product backlog and Sprint Backlog

User Story – Format

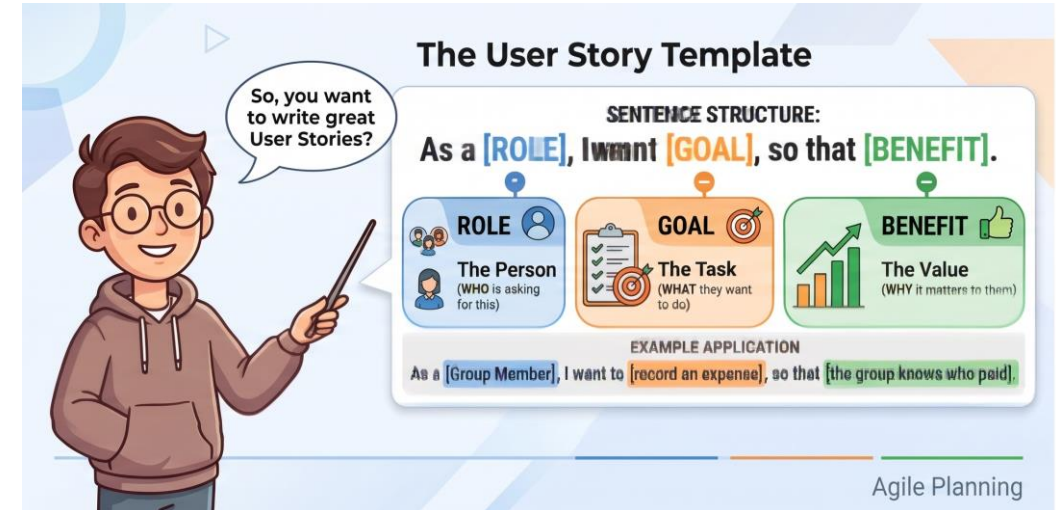
A user story often follows the following **Connextra** format/template:

As a [who] I want to [what] so that [why]

- As a **<type of user>** – this is the **WHO**
 - Who are we building this for? Who is the user?
- I want **<some feature>** – this is the **WHAT**
 - What are we building? What is the intention?
- So that **<some reason>** – this is the **WHY**
 - Why are we building this? What is the value for the customer?

User Story – Format

- As a **[role]**, I want **[goal]**, so that **[benefit]**
- **Role:** who is asking for this
- **Goal:** what they want to do
- **Benefit:** why it matters to them



User Stories

- Example:
 - As an **online shopper**, I want to **add an item to my cart**, so that **I can purchase it**
 - As a **group member**, I want to **record an expense**, so that **my group knows who paid**

Writing Good User Stories (INVEST)

- **Independent**: can be worked on without waiting on another story
- **Negotiable**: details can be discussed, not fixed in stone
- **Valuable**: delivers real value to a user
- **Estimable**: the team can size it
- **Small**: fits comfortably in one sprint
- **Testable**: has a clear way to verify it's done

Your Turn – What do you think of this User Story?

I want to cover the expense when I join a group activity
and store that in the databased persistently

Poor vs Good User Story

I want to cover the expense when I join a group activity and store that in the databased persistently

- Poor User Story: vague, no role, no value stated, technical implementation, other?

As a group member, I want to split an expense unevenly, so that the split matches how group members shared the expense

- A better User Story: specific, testable, and tied to a real need

Acceptance Criteria

- **Acceptance criteria (*satisfaction conditions*)**, provide a detailed scope of end users requirements
- Help the development team understand the **value of the user story** and **set expectations** as to when a team should consider something **done**
- Acceptance Criteria Goals
 - **clarify** what the team should **build** before they start
 - **ensure** everyone has a **common understanding** of the **functionality**
 - help the team members know **when the story is complete**
 - help **verify** the expected behaviour via automated tests

Acceptance Criteria (cont'd)

Acceptance criteria should include:

- **Negative** scenarios of the functionality
- **Functional** and **non-functional** use cases
- **Performance** concerns and guidelines
- What the system/feature intends to do
- **User experience** aspects

Acceptance Criteria (cont'd)

Example:

- As an online banking customer, I want a strong password, so that my credit card information is secure

Acceptance Criteria:

- The password must be at least eight (8) characters
- The password must contain at least one character from each of the following groups:
 - lower case alphabet
 - upper case alphabet
 - digit
 - special characters (!, @, #, \$, %, ^, &, *)

User Stories

- Keep them short
- Keep them simple
- Write them from the **user perspective**
- Make the **value/benefit** of the story clear
- Describe only **one piece of functionality**
- Write stories as a **team**
- Use **acceptance criteria** to show a **Minimum Viable Product (MVP)**, that is, is a **working** and **usable** product

Scrum – Planning, Estimation, and Monitoring

Scrum – Planning and Iteration Estimation

- PO works with the customer to set the **priority of PB items**
- In an iteration, the team divides the PB items into **individual tasks**
- The dev. team **defines tasks and estimates the size/effort** for each item
 - Product Owner/customers are not allowed to change the estimates
- The list of tasks is flexible
- Dev. team tracks all “tasks” on a Task Board (To Do, in-progress, Done)
- Dev. team tracks progress with a burn-down chart

Estimation – Size of User Story

– Ideal days

- The amount of time a user story will take to develop
- Express the estimate as a whole (i.e., 2 ideal days)

– Story points

- Metric for the size of a user story, feature, or other piece of work
- A point value to each item is assigned
- Estimation scale:
 - *Fibonacci series; 1, 2, 3, 5, 8, ...*
 - *Subsequent number as twice the number that precedes it: 1, 2, 4, 8, ...*

Estimation Technique – Planning Poker

- Gamified technique to estimate effort/relative size of development in Agile development
- Agile teams to estimate is by playing planning poker

[Agile Estimating and Planning: Planning Poker - Mike Cohn](#)

Sprint Planning – Story Points

- Start with the most valuable user stories from the product backlog
- Take a story in that list (ideally the smallest one)
- Discuss with the team whether that estimate is accurate
- Keep going through the stories until the team have accumulated enough points to fill the Sprint

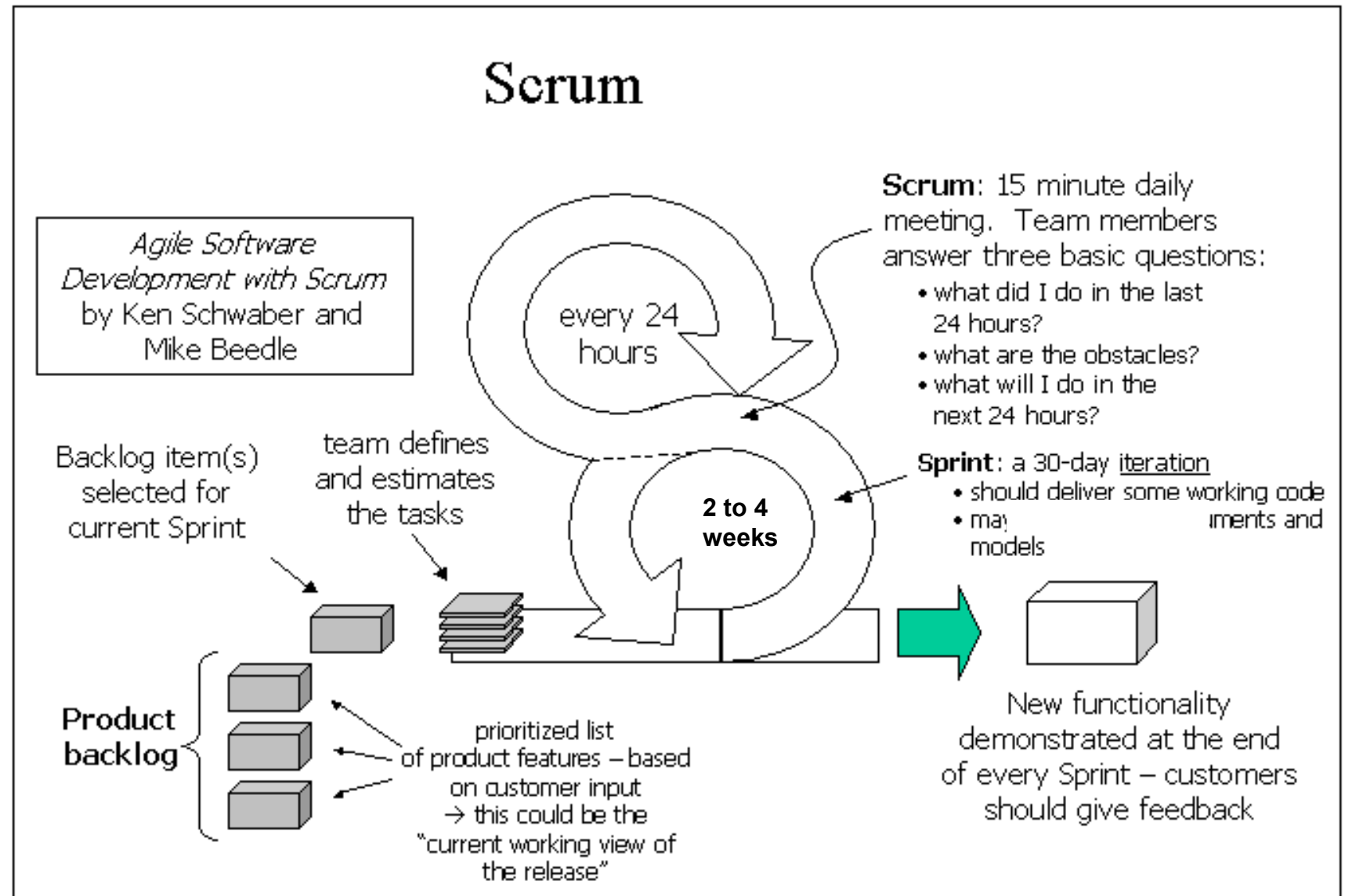
User Story Estimation – T-Shirt Sizing Method

- Uses sizes: **XS, S, M, L, XL**
- **Simple, fast, and beginner-friendly**
- No false precision; good for early-stage teams
- This course's **standard method** for story point examples

Example: A Complete User Story

- **Story:** As a group member, I want to record an expense with a payer, amount, and split, so that the group knows what everyone owes
- **Acceptance criteria:**
 - Expense has a payer, an amount, and a split method
 - Equal split divides the amount evenly across all members
 - Custom split allows entering each member's share
 - Amounts that don't divide evenly are handled without losing or gaining money
- **Story point:** size-M
- In GitLab: one Issue, four Tasks for the criteria, `size-M` label, assigned to the Sprint 1 milestone

Summary of Scrum



Scrum – Tracking and Monitoring Progress

Burndown chart, Velocity, The Sprint/Task board

Planning – Running a Sprint Using SPs, Tasks, and a Task Board

First half of the Sprint planning

- Story points and velocity to figure out what will go into the Sprint

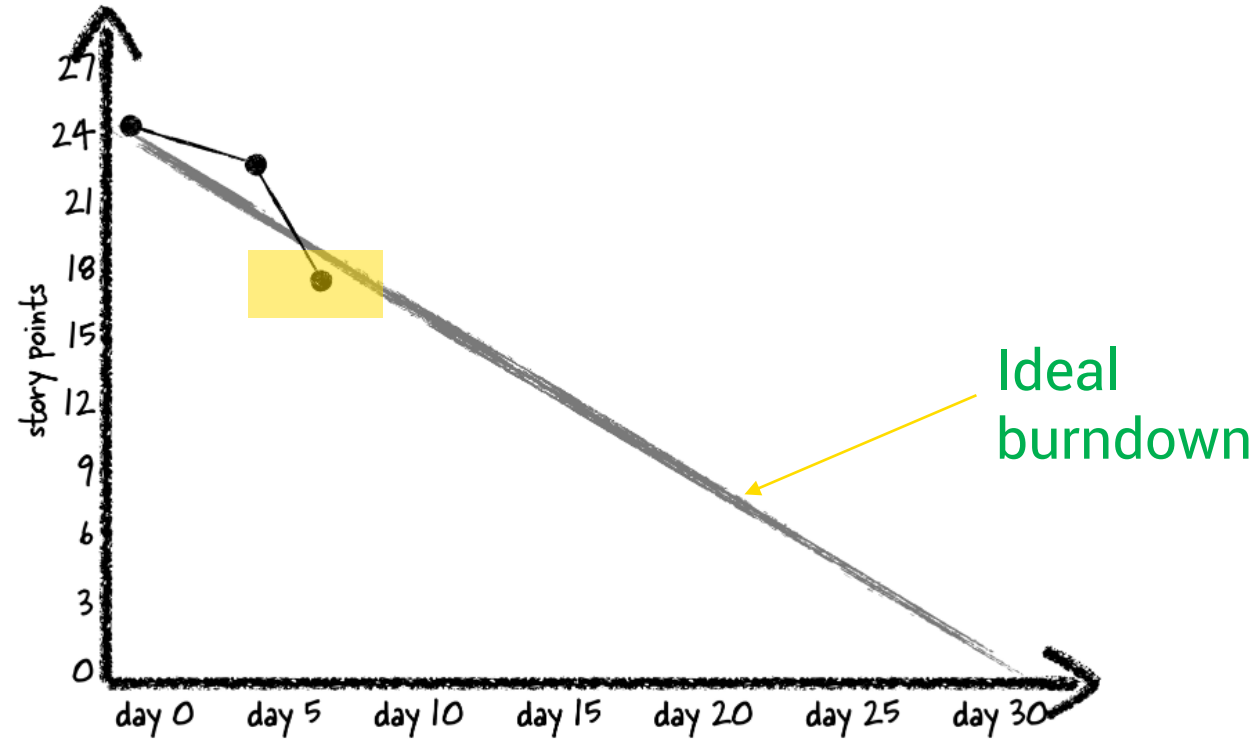
Second half of the Sprint planning

- Plan out the actual work for the team is to add cards for individual tasks
- Tasks can be written code, create design and architecture, and all those other things that teams really do every day to build and release software

Burndown Charts

- Tracks the completion of development work throughout the Sprint;
- Should be visible to everyone in the team (e.g., whiteboard, wall chart, online tool)
- First half of the Sprint planning
 - Story points and velocity to figure out what will go into the Sprint
- Good estimation and planning should help the team to burn stories relatively with similar pace

Burndown Charts based on Story Points



Two stories worth 7 points burned off

Estimating Progress – Velocity

- A measure of a team's rate of progress
- Sum the number of story points assigned to each user story that the team completed during the iteration

Example:

- A team completed 3 stories, 5 SPs each $\rightarrow$ velocity = 15

Tools for Tracking Progress

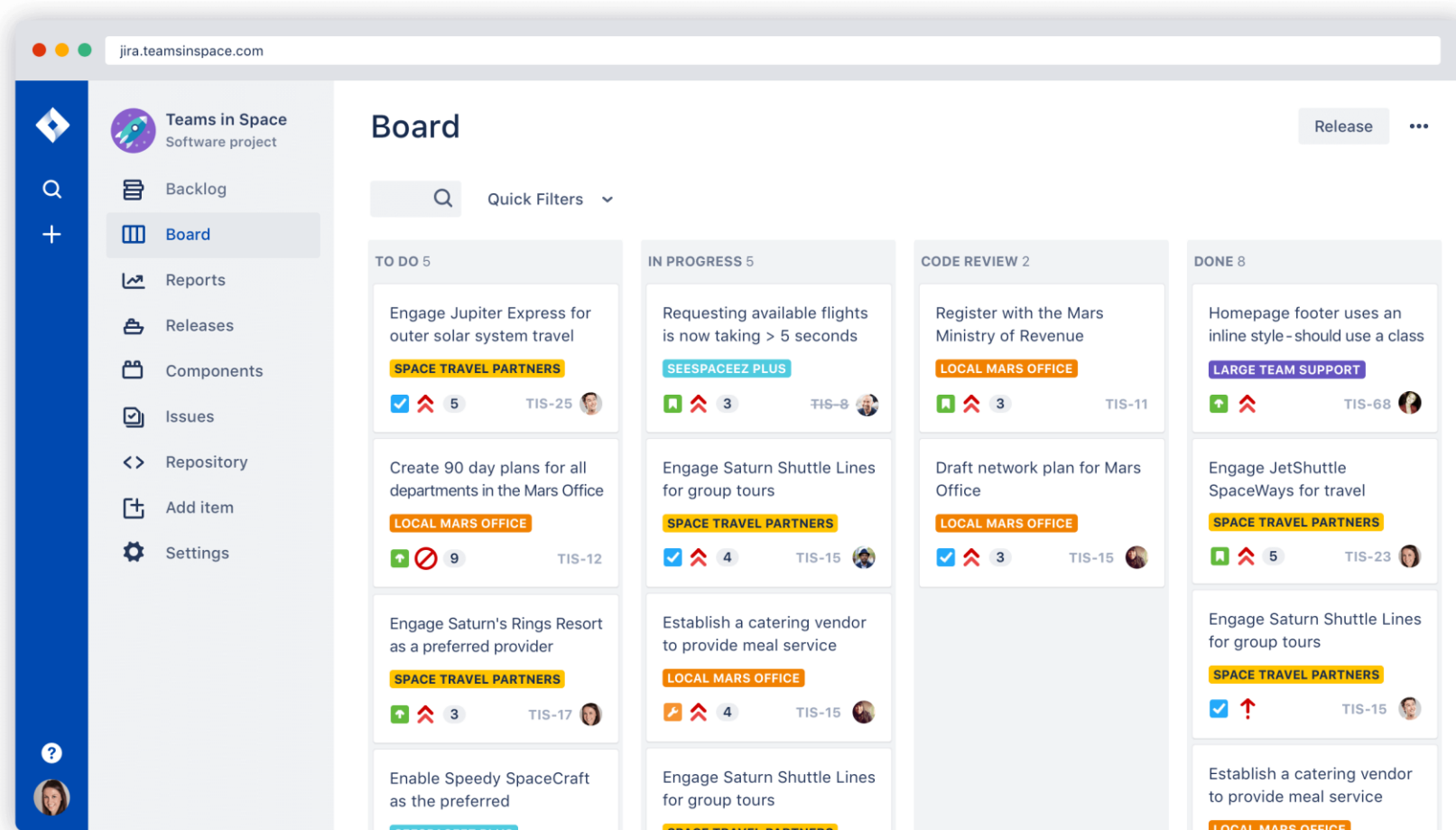
Burndown chart, The Task board

Tool Support for Agile SW Development

- JIRA is a software tool for planning, tracking and managing software development
 - Supports different agile methodologies including Scrum and Kanban
- Jira supports Scrum Sprint planning, stand ups (daily scrums), Sprints and retrospectives
- Including backlog management, project and issue tracking, agile reporting
 - E.g., Burndown and velocity charts, Sprint report
- Scrum boards visualize all the work in a given Sprint

<https://www.atlassian.com/software/jira/agile>

Jira Agile – Scrum Board



<https://www.atlassian.com/software/jira/agile>

Jira Agile – Sprint Planning

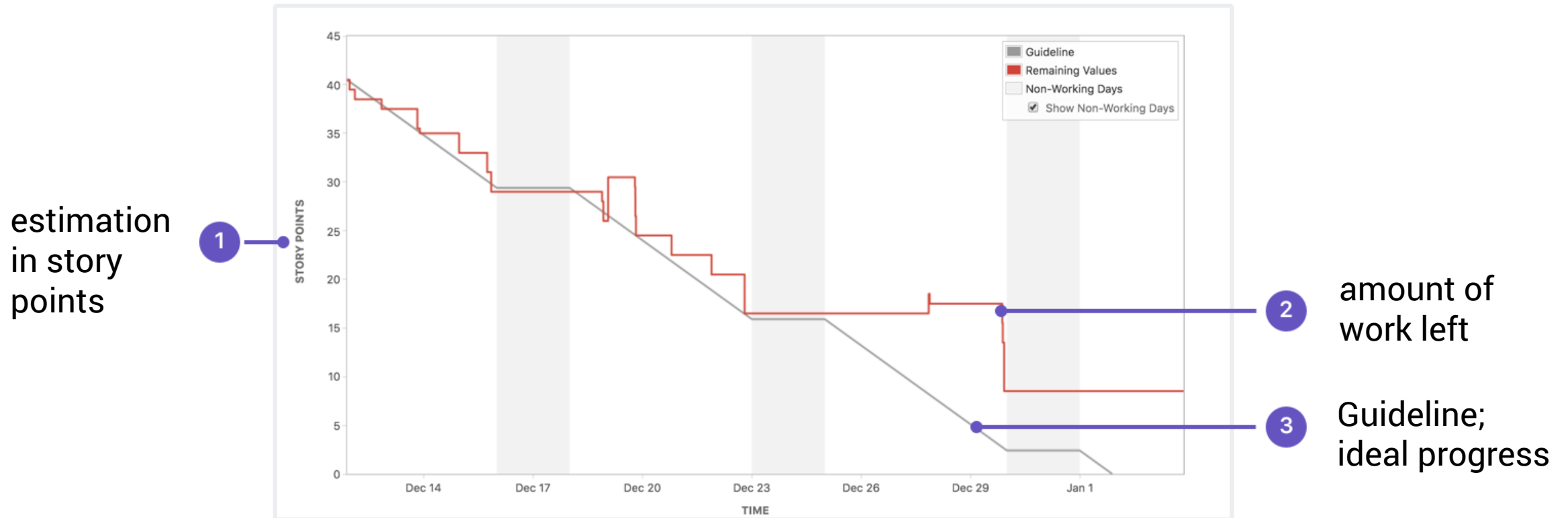
The screenshot displays the Jira Agile Backlog for the 'Teams in Space' project. The interface is divided into several sections:

- Left Sidebar:** Contains navigation links for Backlog, Agile board, Releases, Reports, All issues, Components, and Add-ons. Below these are Project Shortcuts including Mars Team HipChat Room, Space Station Dev Roadmap, Teams in Space Org Chart, Orbital Spotify Playlist, and Hyperspeed Bitbucket Repo.
- Backlog Header:** Shows the project name 'Teams in Space' and a 'Configure' button.
- Quick Filters:** Includes Product, Recently updated, Only my issues, Server, and UI.
- EPICS:** A list of project components such as SeeSpaceEZ Plus, Large Team Support, Space Travel Partners, Summer Saturn Sale, Afterburner Plus, Local Mars Office, Hyper-speed shuttles, New launch platforms, Delicious Space Nutrition, and Spacetainment.
- Sprint 1:** Labeled 'Sprint 1' with 14 issues. It includes a 'Start sprint' button.
- Sprint 2:** Labeled 'Sprint 2' with 6 issues. It shows a start date of 10 Aug 2015 and a release date of 9 Oct 2015. A 'Start sprint' button is also present.
- Issue List:** A table of issues with columns for status, priority, key, summary, assignee, and count. Issues include TIS-25, TIS-37, TIS-9, TIS-7, TIS-10, and TIS-18, assigned to various teams like SeeSpaceEZ Plus, Large Team Support, Local Mars Office, and Space Travel Partners.
- Backlog:** A section showing 49 issues in the backlog, with a 'Create sprint' button.

<https://www.atlassian.com/software/jira/agile>

Burndown Charts

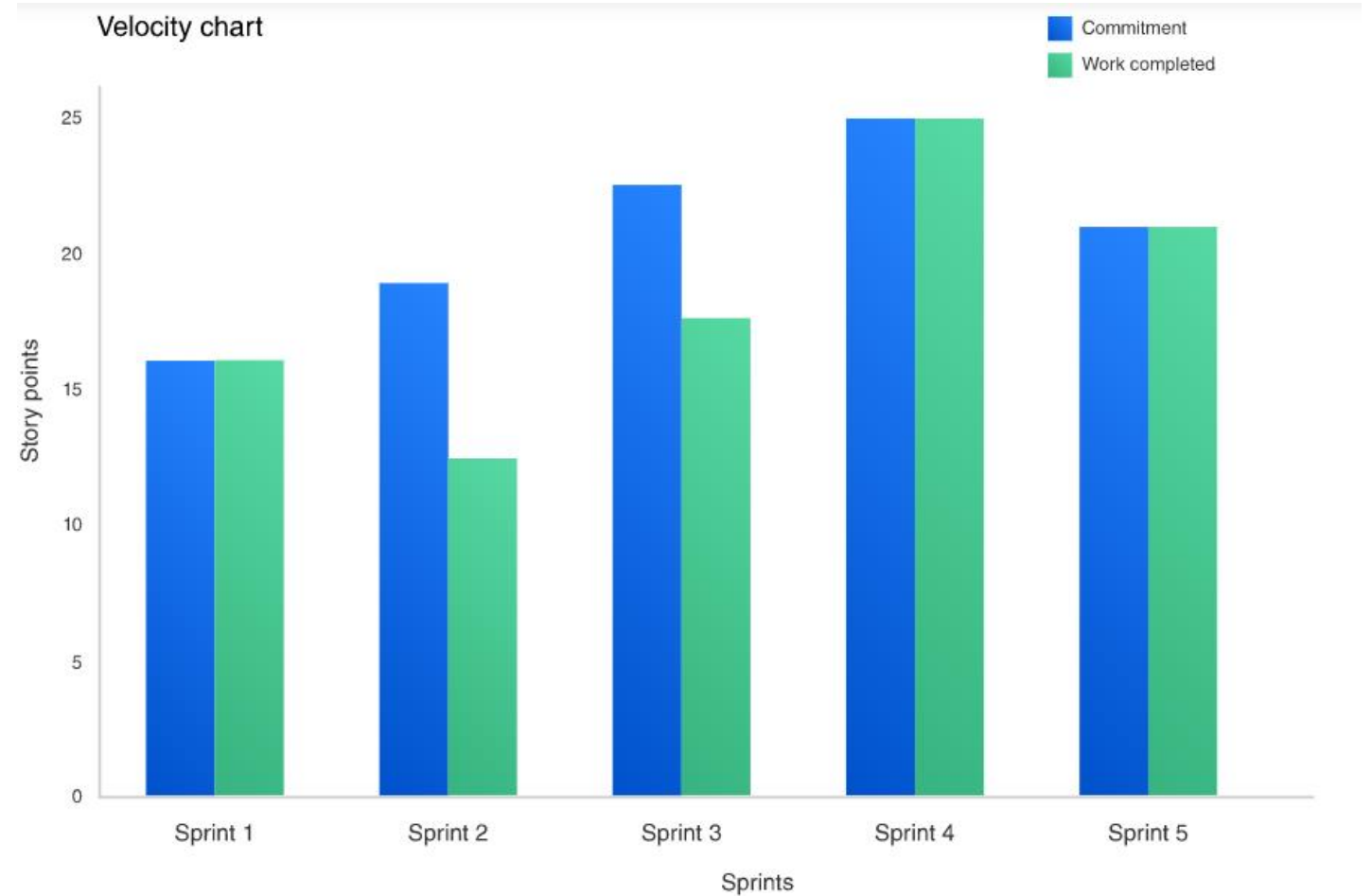
- “**Burndown**” chart tracks the amount of estimated effort remaining in a sprint
 - Maintained daily by the Scrum Master



<https://www.atlassian.com/agile/tutorials/burndown-charts>

Velocity Chart

- *Velocity* tracks the amount of work from Sprint to Sprint
Velocity chart graphically shows project/team's velocity



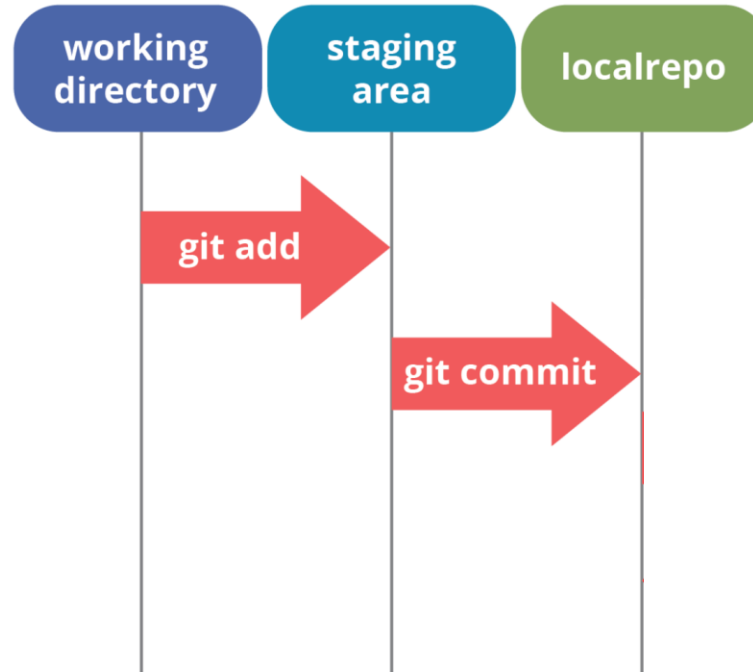
<https://confluence.atlassian.com/jirasoftwareserver/velocity-chart-938845700.html>

Version Control – Remote Collaboration

Git / GitLab Collaboration (revisit)

Version Control – Local and Remote

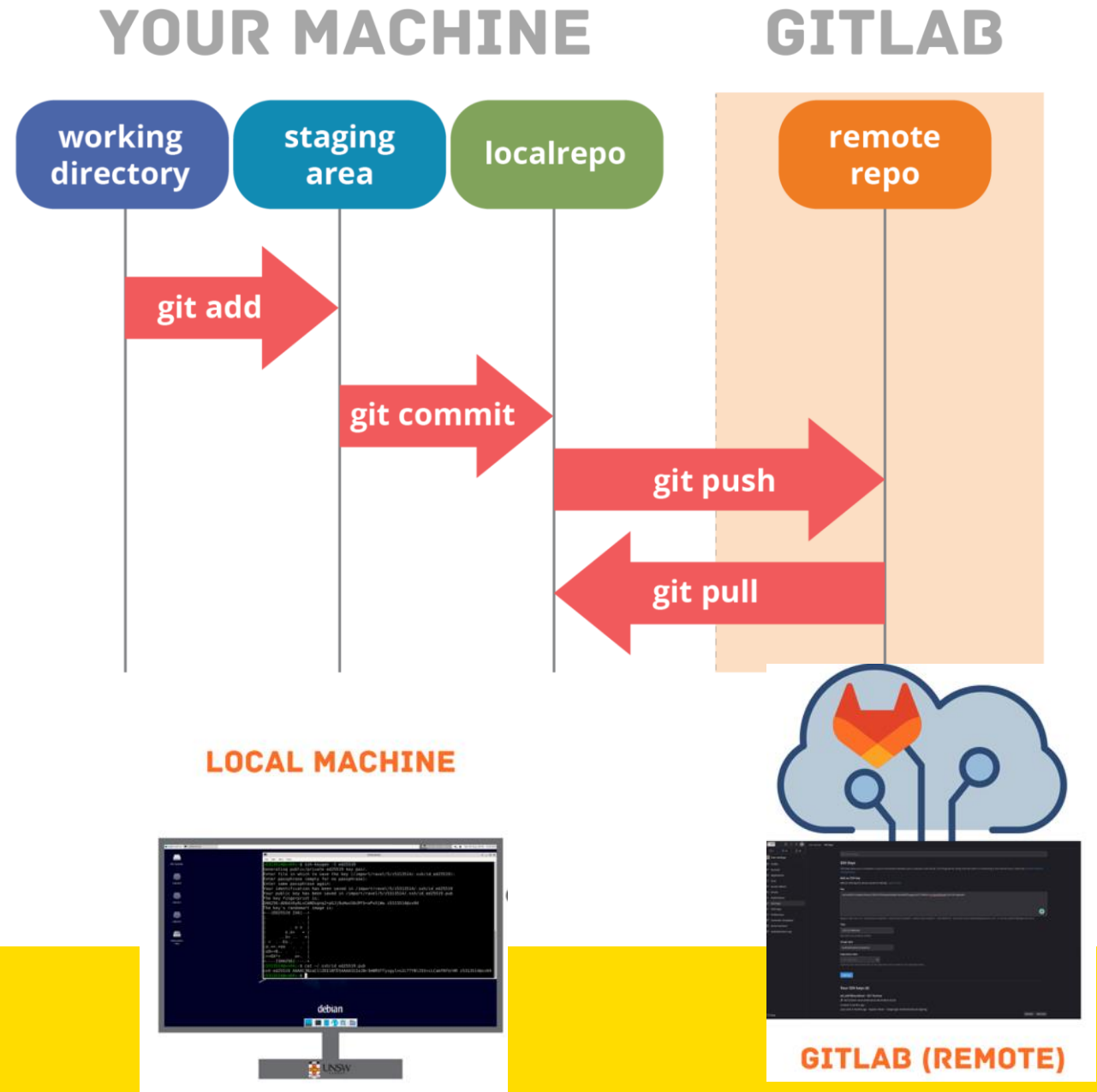
YOUR MACHINE



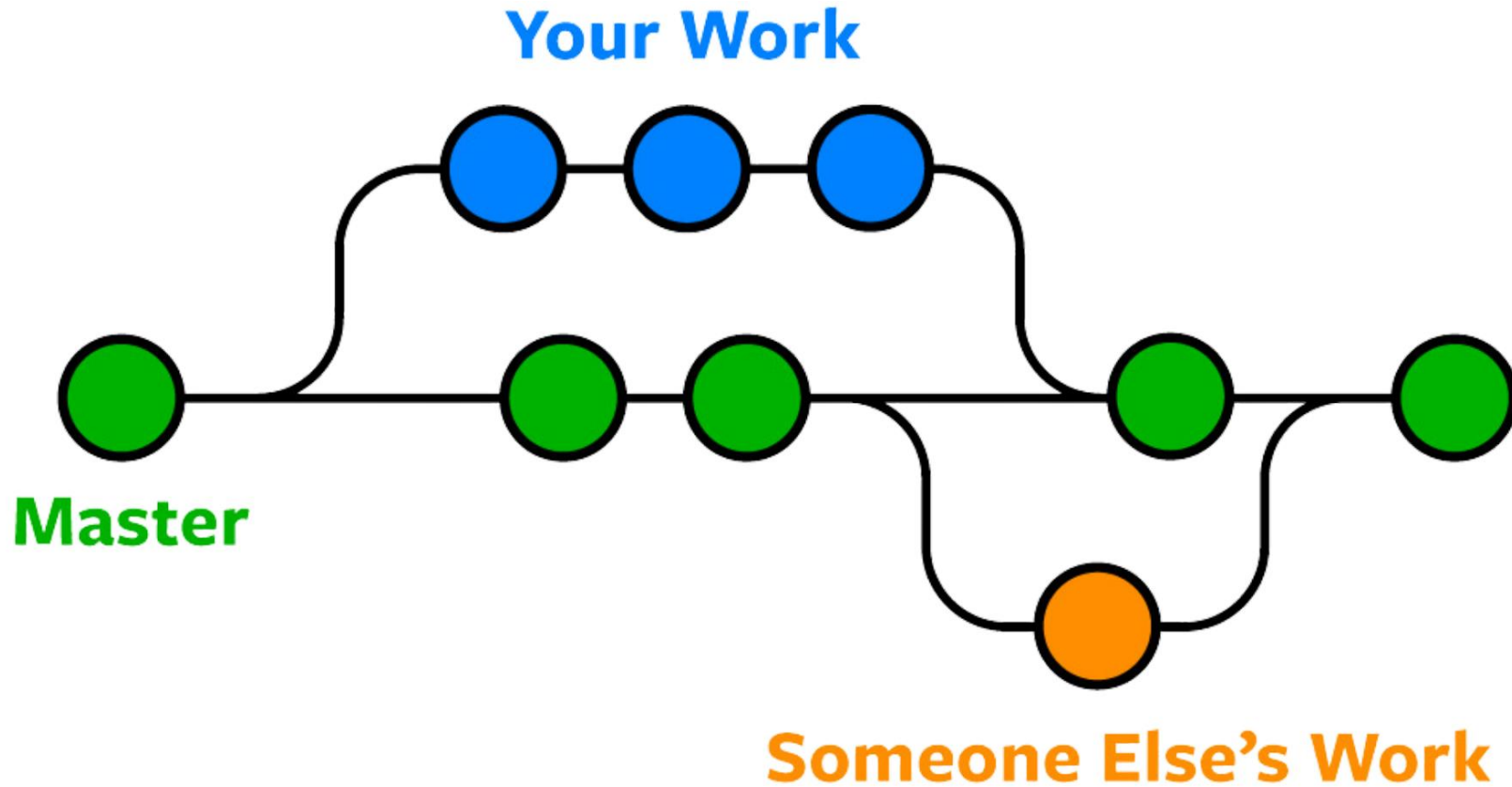
LOCAL MACHINE



Version Control – Local and Remote



Version Control – Branching and Merging



A branch lets you work on the same files without disturbing each other.

References

- Andrew Stellman, Margaret C. L. Greene 2014. Learning Agile: Understanding Scrum, XP, Lean and Kanban (1st Edition). O'Reilly, CA, USA.
- Ian Sommerville. 2016. Software Engineering (10th ed.) Global Edition. Pearson, Essex England
- Agile Alliance. <https://agilealliance.org/>
- Agile Manifesto: <https://agilemanifesto.org/>
- Atlassian Tutorials. <https://www.atlassian.com/agile/tutorials/>

Thank you!